A simulação consiste em dois componentes principais:

1. **app.py**: Representa uma aplicação web que analisa constantemente seu estado e faz previsões sobre a necessidade de alocação de recursos.
2. **stress_test.py**: Simula picos de tráfego para testar a robustez da aplicação web.

Código da Aplicação Web (app.py)

O `app.py` implementa uma aplicação Flask que prevê o estado da aplicação web a cada dois segundos, simulando um aumento de carga. Ele usa um modelo de Random Forest treinado para essa finalidade.

```python
from flask import Flask, request, jsonify
import joblib
import pandas as pd
import threading
import time
import numpy as np

app = Flask(__name__)

# Carregar o modelo salvo
model = joblib.load('random_forest_model.pkl')

# Carregar o dataset completo para extrair exemplos reais
google = pd.read_csv('datasets/google.csv')
google['time'] = pd.to_datetime(google['time'], unit='ns')
google.set_index('time', inplace=True)

# Preencher valores ausentes apenas nas colunas numéricas
numeric_cols = google.select_dtypes(include=[np.number]).columns
google[numeric_cols] = google[numeric_cols].fillna(google[numeric_cols].mean())

# Extrair valores de memória dos dicionários em average_usage e maximum_usage
def extract_memory_value(value):
    if isinstance(value, str):
        value_dict = eval(value)
        return value_dict.get('memory', np.nan)
    return value

google['average_usage_memory'] = google['average_usage'].apply(extract_memory_value)
google['maximum_usage_memory'] = google['maximum_usage'].apply(extract_memory_value)
google['resource_request_memory'] = google['resource_request'].apply(extract_memory_value)

# Selecionar um exemplo de sucesso (aplicação saudável) como estado atual
current_state = google[google['event'] == 'FINISH'][['scheduling_class', 'priority', 'assigned_r

# Função para prever o estado da aplicação
def predict_application_state(input_data):
    prediction = model.predict(input_data)
    return prediction

# Função para atualizar o estado da aplicação e enviar relatório
def update_state_and_report():
    global current_state
```

```
    while True:
        input_data = pd.DataFrame([current_state])
        prediction = predict_application_state(input_data)
        state_message = "Estado desconhecido"

        if prediction[0] < 1:
            state_message = 'A aplicação precisa de mais recursos: {:.2f}'.format(prediction[0])
        elif prediction[0] < 2:
            state_message = 'A aplicação está em risco: {:.2f}'.format(prediction[0])
        else:
            state_message = 'A aplicação está em estado saudável: {:.2f}'.format(prediction[0])

        print(state_message)

        # Simulando aumento de carga
        current_state["average_usage_memory"] += 0.1
        current_state["maximum_usage_memory"] += 0.1

        # Enviar relatório a cada 2 segundos
        time.sleep(2)

@app.route('/predict', methods=['POST'])
def predict():
    global current_state
    data = request.json
    input_data = pd.DataFrame(data)
    prediction = predict_application_state(input_data)
    response = []
    for pred in prediction:
        if pred < 1:
            response.append('A aplicação precisa de mais recursos: {:.2f}'.format(pred))
        elif pred < 2:
            response.append('A aplicação está em risco: {:.2f}'.format(pred))
        else:
            response.append('A aplicação está em estado saudável: {:.2f}'.format(pred))
    return jsonify(response)

if __name__ == '__main__':
    # Iniciar a thread para atualizar o estado e enviar relatórios
    threading.Thread(target=update_state_and_report, daemon=True).start()
    app.run(host='0.0.0.0', port=5000)`
```

Simulação de Picos de Tráfego (stress_test.py)

O `stress_test.py` simula um pico de tráfego enviando múltiplas requisições simultâneas para a aplicação Flask.

```python
import requests
import json
from concurrent.futures import ThreadPoolExecutor
import pandas as pd
import numpy as np

# Carregar o dataset completo para extrair exemplos reais
google = pd.read_csv('datasets/google.csv')
google['time'] = pd.to_datetime(google['time'], unit='ns')
google.set_index('time', inplace=True)

# Preencher valores ausentes apenas nas colunas numéricas
numeric_cols = google.select_dtypes(include([np.number]).columns)
google[numeric_cols] = google[numeric_cols].fillna(google[numeric_cols].mean())

# Extrair valores de memória dos dicionários em average_usage e maximum_usage
def extract_memory_value(value):
    if isinstance(value, str):
        value_dict = eval(value)
        return value_dict.get('memory', np.nan)
    return value

google['average_usage_memory'] = google['average_usage'].apply(extract_memory_value)
google['maximum_usage_memory'] = google['maximum_usage'].apply(extract_memory_value)
google['resource_request_memory'] = google['resource_request'].apply(extract_memory_value)

# Selecionar exemplos de falha para o teste de estresse
test_data_fail = google[google['event'] == 'FAIL'][['scheduling_class', 'priority', 'assigned_me

# URL da aplicação Flask
url = 'http://localhost:5000/predict'

# Função para enviar uma requisição
def send_request(data):
    response = requests.post(url, headers={"Content-Type": "application/json"}, data=json.dumps(
    print(response.json())

# Número de requisições simultâneas
num_requests = 100

# Usar ThreadPoolExecutor para enviar requisições simultâneas
with ThreadPoolExecutor(max_workers=num_requests) as executor:
    futures = [executor.submit(send_request, data) for data in test_data_fail for _ in range(nur
```

```
# Esperar todas as requisições completarem
for future in futures:
    future.result()`
```

Resultados da Simulação

Saída de `app.py`:

```
A aplicação está em risco: 1.93
A aplicação está em risco: 1.93
A aplicação está em risco: 1.93
A aplicação está em risco: 1.93
...
127.0.0.1 - - [06/Jun/2024 14:25:41] "POST /predict HTTP/1.1" 200 -
...`
```

Saída de `stress_test.py`:

```
['A aplicação está em risco: 1.10']
['A aplicação está em risco: 1.10']
['A aplicação está em risco: 1.10']
['A aplicação está em risco: 2.00']
...`
```

Análise dos Resultados

O modelo de Random Forest foi capaz de prever o estado da aplicação com base nas métricas fornecidas. Durante o teste de estresse, a aplicação repetidamente indicou que estava em risco, conforme previsto pelo modelo. Este comportamento é esperado quando a carga na aplicação aumenta, simulando um pico de tráfego. O uso de múltiplas requisições simultâneas no `stress_test.py` foi eficaz para testar a robustez do modelo e da aplicação.